Capstone Report: Refresh / Content Opportunity Scoring Pipeline

Researcher	Eman Shahzad
Platform	FlyRank
Cohort	2026

Why I chose this

Websites are full of pages that quietly lose traffic. Nobody has time to rewrite all of them, so the real question is where to start. I chose the Refresh / Content Opportunity Scoring lane because it turns search data into a clear, practical answer: a ranked list of pages worth a second look, with a reason attached to each one.

My research question is simple. **Which pages should a content team review for a refresh first, based on how each page has been trending?**

How I will build it

1. **Get set up.** I will request access to the FlyRank dataset on Hugging Face, create a read token, and keep it out of my repo. My repo will have `work/` for notebooks, `docs/` for the paper, and `submission/paper_url.txt` for the final link.

2. **Get to know the data.** Using the DuckDB pattern from starter notebook 03, I will find the page, date, clicks, impressions, CTR and position columns, check the date range for gaps, and plot a few growing, declining and flat pages. I will also write down anything I exclude, and why.

3. **Define what "needs review" means.** I will split time into a past window for features and a later window for the label. A page needs review if its clicks fall by 20% or more, among pages with enough traffic to matter. I will choose these thresholds once and not adjust them to make the results look better.

4. **Build features from the past only.** Click and impression trends, CTR and position changes, and how bumpy a page's traffic is. Nothing from the label window is allowed in.

5. **Start with a simple baseline.** The first rule will be "if clicks fell recently, they will keep falling." A model is only useful if it beats something this simple.

6. **Train and test honestly.** I will begin with Logistic Regression because it is easy to explain, then try one stronger model. I will use a time-aware split, compare model and baseline on the same test set using precision, recall and PR-AUC, and write down every leakage check I run.

7. **Turn scores into actions.** Each page gets a priority of probability times traffic at stake, so a likely decline on a busy page ranks above one on a quiet page. Each page also gets a reason code, such as `CLICKS_FALLING` or `POSITION_SLIPPING`, mapped to a next step like refresh the content, review the title and meta description, or just monitor.

8 **Publish the paper.** I will write it with all nine required sections, deploy it on GitHub Pages, and put the exact URL as the only line in `submission/paper_url.txt`.

Keeping it honest and safe

I will describe results as observed and directional, and use phrases like “decision support.” I will not claim to prove how Google’s algorithm works, or that a refresh caused a change in traffic. Pages will be anonymized, and I will keep client names, domains, URLs, private queries, tokens and raw exports out of the repo and the paper.

The data credit will read “Built on the FlyRank ML Internship dataset,” linking to flyrank.ai.

What I will deliver

A live research paper, a GitHub repo with every assignment notebook plus the capstone notebook, and a ranked table of pages with reason codes that someone could act on straight away.

My checklist

- [ ] Access granted and repo set up
- [ ] Label, features and baseline done
- [ ] Model validated and leakage checked
- [ ] Ranked table with reason codes
- [ ] Paper live, URL saved in submission/paper_url.txt
- [ ] Public-safety check done and repo URL submitted